Fullstack Product Engineer case study

Evidence-first Fit/Gap review

An explainable comparison that keeps missing evidence visible, preserves human rating provenance, and protects access between organizations.

28 October 2026 | Rails + PostgreSQL + React + TypeScript

What changes	Why it matters
One typed comparison contract	Required and assessed levels are readable together; every vacancy requirement remains visible.
Uncertainty and human provenance	Missing assessment is not presented as a low rating. Assessor overrides and AI confidence remain distinct.
Authenticated tenant boundary	Portfolio, comparison, export, and override paths constrain access through the owning session.
Deterministic calculation	This workflow no longer depends on model availability or repeated background generation.

Review artifacts

Open the GitHub Pull Request

Watch the 3-minute demonstration

The demonstration uses synthetic records and Indonesian subtitles. Code and documentation were developed with AI assistance; the candidate must review and understand the implementation before the live technical defense.

Scope: the post-interview portfolio-to-vacancy review. Live audio interviews and upstream Gemini portfolio generation are outside the verified slice.

Steps 1-2

Setup, product context, and people affected

Starting point

Cloned the supplied repository at b836d02f45c6300b238b98ad50e66ead706723b6 and created a feature branch from main. Read both service READMEs, schema, controllers, services, React report and product wiki PRDs. Static inspection ran in parallel with Windows bootstrap; workflow and database claims were then validated on the running app. Wiki filenames incompatible with Windows were read from Git blobs into safe local filenames.

Local execution

The Windows host initially had Node but no Ruby, PostgreSQL, Redis or Docker on PATH. Prepared a workspace-local Ruby 3.3.2 runtime, native compiler, and a PostgreSQL instance bound to 127.0.0.1:55432. Native dependency compilation needed C17 because the newer compiler default was incompatible with Ruby 3.3 headers. No system database or real candidate data was used.

What the product is for

An assessor reviews an interview-derived skill portfolio, corrects a rating when justified, then compares it against a vacancy. The useful outcome is a defensible next step: accept an assessment, inspect its evidence, or collect missing evidence. The report is not a replacement for a hiring decision.

Person	Decision supported / potential harm
Assessor	Check evidence, explain a correction, and identify the rating that actually drives the comparison.
Recruiter	See which requirements need follow-up instead of mistaking missing measurement for poor skill.
Hiring manager	Compare proficiency against an explicit role requirement and understand uncertainty.
Candidate	Avoid unsupported low ratings and keep interview material within the appropriate organization.

Domain grounding and limits

OPM guidance supports using consistent rating scales, but its fixed-question interview method differs from this adaptive PRD. Only the scale-consistency principle informs this design. The brief identifies Indonesian hiring and UU PDP as context; tenant protection and synthetic artifacts address a concrete privacy boundary. These personas are hypotheses from the brief/wiki, not findings from conducted user interviews.

Step 3

Problems ranked by user impact

Severity definitions: P0 blocks release for unauthorized access; P1 can distort an assessment decision; P2 prevents reliable completion or recovery; P3 is a smaller usability defect.

Priority / type	Observed flaw and impact	Decision
P0 / defect	Unscoped portfolio and override lookups can expose another tenant's assessment material to a permitted assessor with a known ID.	Scope ownership through session; bind tenant to authenticated JWT.
P1 / defect	API expected_level and web required_level disagree. The recruiter sees no required level.	Use the actual API contract and assert visible L3/L4 text.
P1 / defect	Label-keyed aggregation drops repeated requirements; label fallback can merge different IDs.	One row per requirement; ID-first, conservative custom matching.
P1 / insufficient evidence	Culture/hiring prose is requested from skill patterns without behavioral evidence.	Remove unsupported prose from this workflow; require separate human review.
P2 / defect	404 polling requeues generation; regeneration deletes the prior report before replacement.	On-demand deterministic computation and atomic upsert; explicit retry.
P2 / missing specification	Ambiguous matching and confidence after overrides lack a complete policy.	Define null semantics and preserve AI confidence separately from human provenance.
P3 / defect	Silent failures and rigid report layouts impede recovery and small-screen review.	Named navigation, recovery states, wrapping and stacked rows.

Constraint signal to the technical lead

Upstream portfolio generation still coerces invalid model levels with to_i.clamp(1, 5) and replaces skills before all new rows are validated. That separate P1 issue can manufacture low ratings or leave partial data. It must be addressed before treating upstream AI output as validated evidence; this submission does not claim that fix.

Also reconcile the yearless 19 August deadline with the hiring portal and audit remaining tenant paths before a wider release.

Step 4

Solution options and acceptance criteria

Option	Benefit / cost	Trade-off
A. Rename the field	Smallest change; restores a visible required level.	Leaves matching, tenant isolation, uncertainty and failure-loop problems.
B. Deterministic review (chosen)	Fast and explainable; no provider/queue dependency for arithmetic.	Automated culture prose is removed; human behavioral review remains necessary.
C. Durable asynchronous AI pipeline	Could preserve prose with explicit status, retry and leases.	More schema/coordination cost; evidence grounding and provider evaluation still required.

Why B fits this codebase

Comparing a small set of skill levels is deterministic. Removing the external model from that calculation eliminates its latency, timeout and duplicate-generation failure paths. The existing JSONB report and unique portfolio/vacancy index are sufficient. Retain narrative columns for rollback; no destructive schema migration or artificial new column is introduced.

Acceptance criterion	Correct behavior
Typed levels	Integer L1-L5 in API, web and export; invalid/missing values stay null.
Complete requirements	One row per vacancy requirement, including repeated labels and unassessed skills.
Conservative identity	Prefer canonical IDs; do not merge different IDs; ambiguous custom matches stay unresolved.
Explicit provenance	Assessor rating wins; show its source and keep AI confidence correctly labeled.
Protected access	Authenticate role and tenant; scope portfolio/session, vacancy, export and override paths.
Reliable refresh	Read changed overrides/requirements; upsert one report; preserve prior data if writes fail.
Usable states	Loading, retry, empty, partial, long text, mobile, keyboard and export failure are represented.
Honest decision boundary	No missing-data penalty, culture inference, fit percentage or automated hiring recommendation.

Step 5

Implementation and failure behavior

Layer	Responsibility
React report	Fetch the portfolio and comparison once. Explain coverage and rating provenance; filter follow-up; recover on user retry; export current data.
Rails controller	Require assessor/admin role, bind tenant to authenticated claims, constrain records, validate readiness and inputs.
FitGap::Engine	Lock the portfolio, verify tenant equality, read current inputs, run the comparator and atomically upsert the report.
FitGap::Comparison	Pure Ruby rules: ID/label matching, strict levels, override precedence, null reasons and signed delta.
Persistence / export	Existing unique index prevents duplicate report rows. JSON and PDF use current comparisons; PDF embeds a licensed Unicode font.

Designed failure paths

Condition	Behavior
Missing / invalid / ambiguous rating	not_assessed, null level and delta, explicit reason and follow-up guidance.
Repeated or legacy generation jobs	Same deterministic upsert under a portfolio row lock; no new provider request.
Database failure during refresh	Transaction preserves the previous report. Override plus dependent refresh rolls back together.
API/network failure	Visible explanation and retry action; no silent blank report or automatic requeue loop.
Export failure	Keep the comparison visible, show an error, and re-enable export.
Provider timeout/model error	No provider call exists in this selected Fit/Gap path. Upstream portfolio failures remain outside this slice.

Maintainability and rollback

The comparator can be tested without Rails. No new reporting schema is required. Deploy API and web together; the comparison metadata is additive, while removal of unsupported culture prose is an intentional product behavior change. Existing narrative columns and the legacy worker entry point remain, allowing a code rollback without a data-destructive down migration.

Step 5

Verification, seeded fault, and AI review

Executed check	Result / evidence
Rails + PostgreSQL	44 examples passed. Includes auth/tenant isolation, current comparisons, empty data, override behavior, rollback and PDF/JSON export.
React unit tests	10 tests passed for the contract, uncertainty, filtering, loading, errors/retry and export recovery.
Browser contract tests	7 tests passed. Real React/browser with synthetic API fixtures; four widths (320/390/768/1280), long text, keyboard, empty, error and export controls.
Production web build	TypeScript and Vite build passed. This is build evidence, not a claim about live Gemini transport.
Running application demo	Browser uses the real local Rails API with synthetic PostgreSQL records. Shows a human override, current exports, mobile review, simulated network failure/retry and empty configuration.

Seeded fault proof

On scratch branch codex/seeded-fault-proof, commit a1bae27 deliberately changed candidate - expected into expected - candidate. The same 25 comparator examples produced 2 failures. Revert 79e06aa restored 25 passing examples. The fault and revert were fast-forwarded into the feature branch, preserving visible history. No assertions were removed or weakened.

AI verification moment

The initial AI-authored tenant patch scoped by current_tenant_id but did not bind that resolved tenant to authenticated JWT claims. Review found the pre-authentication header fallback. Added verify_authenticated_tenant! and a request regression test that rejects a signed token without a scheme even when a header selects a tenant. This is an actual correction of generated code, not a fabricated model incident.

Evidence distinctions

Mocked frontend/browser tests do not prove database authorization or transaction safety; separate Rails/PostgreSQL specs do. The demonstration uses actual API requests, except for the explicitly simulated network failure. No live-audio interview, fairness study, performance benchmark or real-user usability study is claimed.

Step 5 / visual evidence

Before: required levels disappear

← Fit/Gap Report

Regenerate PDF JSON

Skill Comparison

Skill	Required	Candidate	Result
React		L2	⚠ Gap -1
Testing and debugging		L3	✓ Match
Technical communication		L3	★ Exceeds +1
System design		—	— Not assessed

✓ Match: 1 skill ⚠ Gap: 1 skill ★ Exceeds: 1 skill ✎ = human override applied

Culture & Competency Fit

1 match, 1 above requirement, 1 below requirement, 1 not assessed. Missing assessment is not a skill gap. Review interview evidence before making a hiring decision.

Baseline React report reading the actual API-shaped response. The Required cells are blank because the component reads required_level while the API sends expected_level. The old summary also omits unassessed skills. All displayed records are synthetic.

The baseline UI is served from main-branch source with a separate local dependency cache. The screenshot is a reproduction of the contract mismatch, not a production incident.

Step 5 / visual evidence

After: uncertainty and human provenance

← Back to portfolio

Fit/Gap review

Frontend Engineer - synthetic demo

Refresh comparison | Export PDF | Export JSON

Assessment coverage

Review interview transcript

3 of 4 requirements assessed

3 meet or exceed requirements · 0 below requirement · 1 not assessed

Missing assessment is not a skill gap. This comparison supports human review and does not make a hiring recommendation.

Skill comparison

All (4) | Needs follow-up (2)

Skill	Required	Assessed	Comparison
React AI confidence: high	L3	L4 Assessor override	Above requirement Difference: +1 level
Testing and debugging AI confidence: low	L3	L3 AI rating	Meets requirement Difference: 0 levels
Technical communication AI confidence: high	L2	L3 AI rating	Above requirement Difference: +1 level
System design	L3	Not assessed No usable rating	Not assessed This skill was not assessed. Arrange a follow-up if it matters for the role.

Follow-up includes skill gaps, missing assessments, and low-confidence AI ratings still used in this comparison. Confidence describes the AI assessment; an override identifies the human rating used.

Culture and competency need separate review

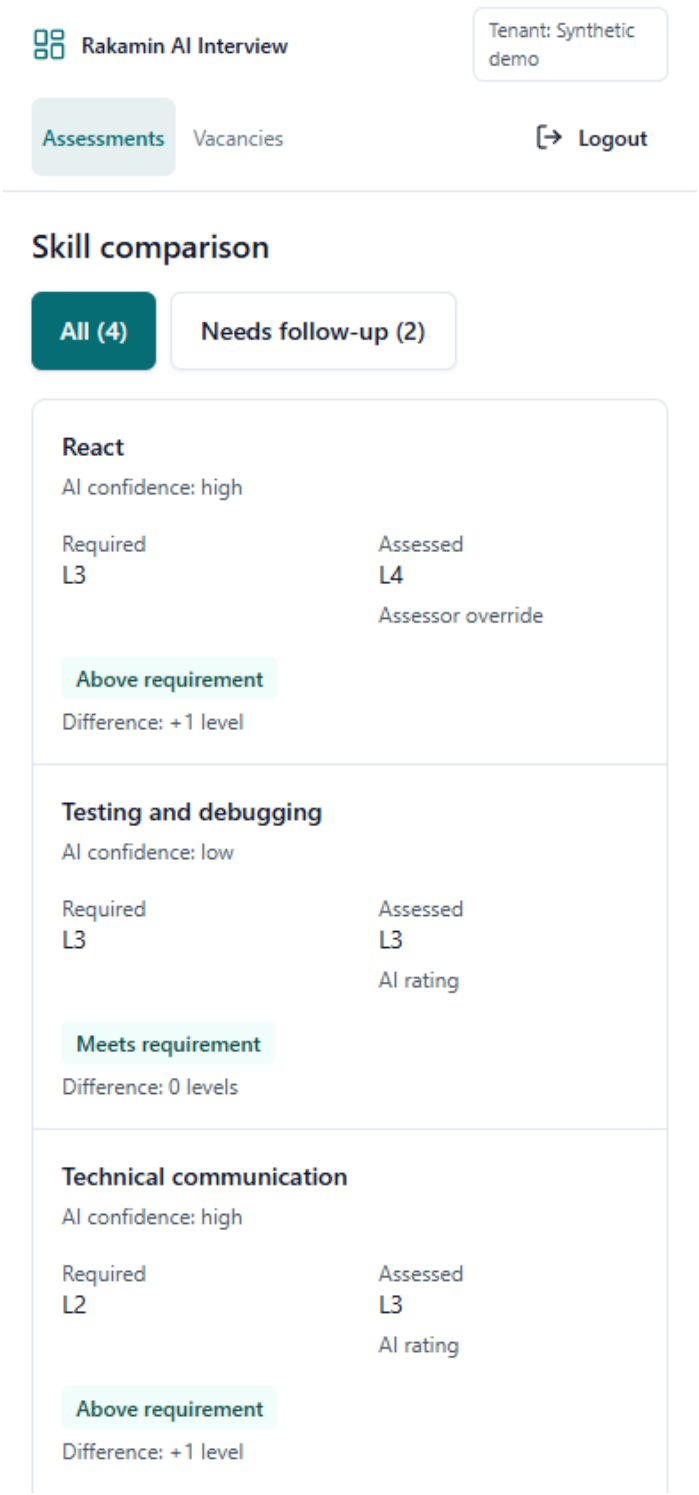
Skill levels alone do not establish culture fit. Review behavioral evidence and the role context with an assessor before drawing a conclusion.

▶ How this comparison works

The running Rails API provides current required and assessed levels, assessor override provenance, AI confidence, and an explicit unassessed requirement. Coverage is shown separately from the comparison outcome; it is not a fit percentage.

Step 5 / visual evidence

A readable mobile review



The comparison reflows

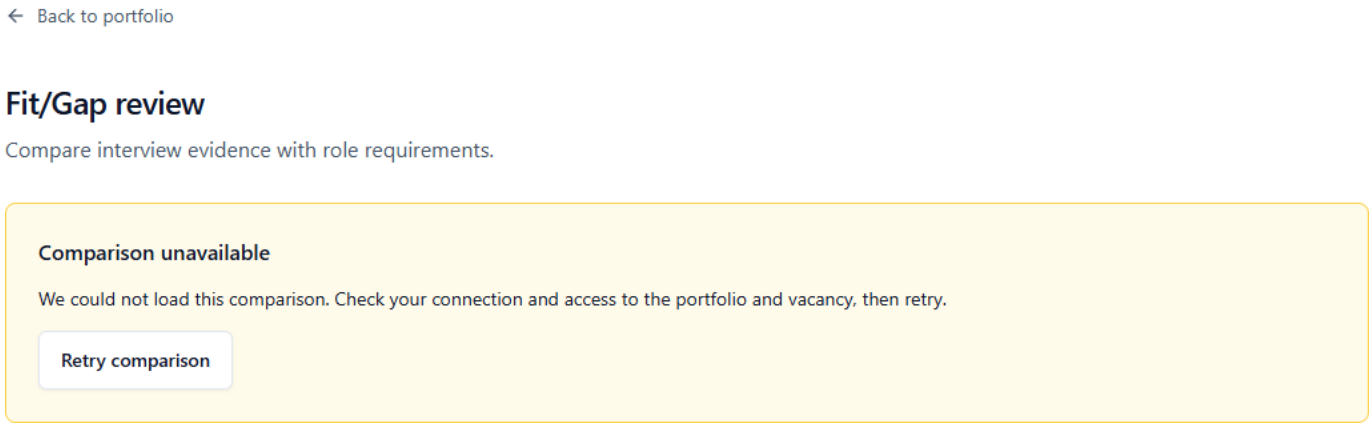
At 390px, each requirement becomes a stacked row. Required and assessed levels, source, confidence and the result remain visible. Long labels wrap without horizontal page scrolling.

The header keeps navigation, active tenant and logout available. Review controls have large touch targets and visible keyboard focus.

Automated browser checks also exercise 320px, 768px and 1280px viewports with long unbroken labels.

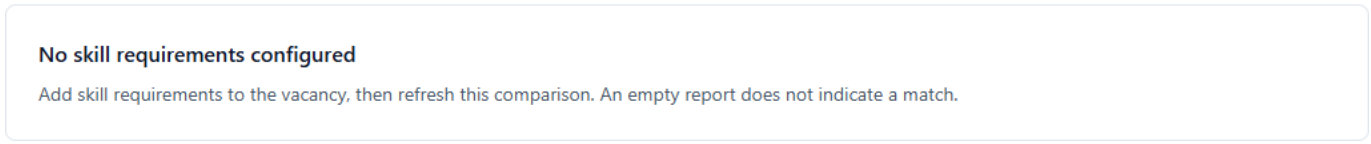
Step 5 / visual evidence

Designed recovery and empty states



A deliberately aborted comparison request reveals a retry action. Removing the simulated failure restores the report without queueing a job. The video shows this recovery against the running Rails API.

Empty configuration



A vacancy with no requirements returns an empty comparison and a clear instruction to configure the vacancy. It does not imply a candidate match. The video includes this state; browser tests also check long unbroken labels across four viewport widths.

Typography, borders and the existing teal identity are retained. Buttons remain keyboard-operable; report controls have large touch targets and visible focus. No decorative statistics, testimonials or additional navigation destinations were invented.

Step 6

Submission and review handoff

Required submission artifacts

1. GitHub Pull Request: fullstack implementation, tests, workflow, decision documents and visible seeded-fault history.

2. 3-minute video walkthrough: synthetic data, real local API requests, and captioned explanations of the chosen product behavior.

3. This single PDF: execution narrative, ranked findings, acceptance criteria, option comparison, engineering proof, screenshots and constraints.

Privacy and data handling

The implemented boundary requires authenticated role/tenant checks and ownership-constrained queries. Artifacts use synthetic data. Local development tokens and fixture connection material stay in ignored files; no real interview transcripts or provider keys are committed. No new model request sends candidate data from this comparison workflow.

The broader deployment still needs explicit decisions about legal basis, notices, retention, data-subject requests and provider transfers. This bounded engineering change is not an application-wide compliance certification.

What remains outside this submission

Upstream portfolio generation validation/atomicity, live voice/Gemini transport, application-wide tenant audit, production load testing, and direct user validation. The deadline printed in the brief has no year; reconcile it with the hiring portal before uploading.

Sources

Baseline repository and service documentation

Product wiki PRD 01 and PRD 02

OPM: Structured Interviews (rating-scale principle only)

Also consulted: the supplied four-page case-study brief. OPM guidance is a methodological reference, not Indonesian regulation. Product/persona observations are reasoned hypotheses unless identified as executed application checks.

Live technical defense

The candidate should be able to explain the matching policy, null semantics, override precedence, trust boundary, transaction behavior, rejected alternatives, seeded fault and remaining risks. A Bahasa Indonesia preparation guide is included in assessment/INTERVIEW_PREP.md.